KRISTU JAYANTI INSTITUTE OF TECHNOLOGY

Department of Computer Applications

ADBMS PROJECT PROPOSAL

Digital Wallet & Payment Management System

Submitted by

LITHIKSHA V

Reg. No: 26MCAB56

Digital Wallet & Payment Management System

Course: Advanced Database Management System Practical
Database: MySQL

Project Description

The Digital Wallet & Payment Management System (DWPMS) will be developed as a database project for storing and managing user details, digital wallets, linked bank accounts, merchants, transactions and payment requests. Self-created dataset will be used to complete database design, populated and tested using the concepts covered in the ADBMS practical.

Objectives

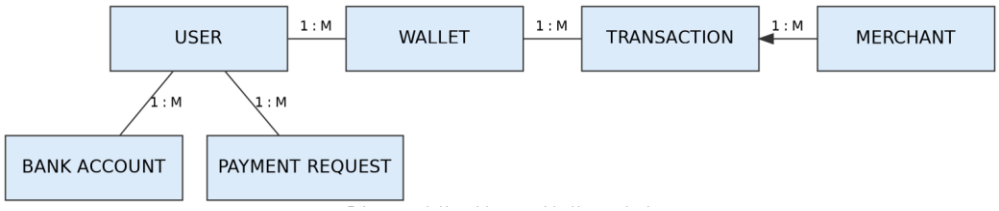
1. To enable the design of a structured dataset using proper data modelling techniques.
2. To develop practical skills in SQL for database creation, manipulation and querying.
3. To enhance analytical thinking through multi-phase database development.

Phase 1 - Data Modelling and Dataset Design

1. **Dataset:** A small DWPMS dataset containing users, wallets, bank accounts, merchants, transactions and payment requests. The same data will be used in all later phases.
2. **Entity and attribute identification:** USERS, WALLETS, BANK_ACCOUNTS, MERCHANTS, TRANSACTIONS and PAYMENT_REQUESTS as the main entities, with attributes required to store their respective details.
3. **ER modelling:** USER-WALLET, USER-BANK_ACCOUNT, USER-PAYMENT_REQUEST, WALLET-TRANSACTION and MERCHANT-TRANSACTION will be modelled mainly as 1:1 and 1:M relationships.
4. **Relational schema:** Convert the ER model into tables. Primary keys will identify records and foreign keys will connect related tables.
5. **Normalization:** user, bank account, merchant and transaction details will be stored separately so that repeated data is reduced and the schema remains in 3NF.

ER Diagram

Primary relationships used in the project



Tables and Data Types

Table	Fields with Data Type	Relationship
USERS	user_id INT PK AUTO_INCREMENT name VARCHAR(100) NOT NULL email VARCHAR(100) UNIQUE phone VARCHAR(15) UNIQUE password_hash VARCHAR(255) kyc_status VARCHAR(20)	USERS 1:1 WALLETS USERS 1:M BANK_ACCOUNTS USERS 1:M PAYMENT_REQUESTS
WALLETS	wallet_id INT PK AUTO_INCREMENT user_id INT FK balance DECIMAL(12,2)	FK user_id -> USERS WALLETS 1:M TRANSACTIONS

Table	Fields with Data Type	Relationship
	status VARCHAR(20) updated_at DATETIME	
BANK_ACCOUNTS	account_id INT PK AUTO_INCREMENT user_id INT FK account_number VARCHAR(20) UNIQUE ifsc_code VARCHAR(11) bank_name VARCHAR(100) is_primary BOOLEAN	FK user_id -> USERS
MERCHANTS	merchant_id INT PK AUTO_INCREMENT business_name VARCHAR(150) category VARCHAR(50) merchant_wallet_id INT FK	FK merchant_wallet_id -> WALLETS MERCHANTS 1:M TRANSACTIONS
TRANSACTIONS	txn_id BIGINT PK AUTO_INCREMENT sender_wallet_id INT FK receiver_wallet_id INT FK merchant_id INT FK amount DECIMAL(12,2) txn_type VARCHAR(20) status VARCHAR(20) created_at DATETIME	FK sender/receiver_wallet_id -> WALLETS FK merchant_id -> MERCHANTS
PAYMENT_REQUESTS	request_id BIGINT PK AUTO_INCREMENT sender_id INT FK receiver_id INT FK amount DECIMAL(12,2) status VARCHAR(20) created_at DATETIME	FK sender_id/receiver_id -> USERS

Relational Schema: USERS(user_id) -> WALLETS(user_id), BANK_ACCOUNTS(user_id), PAYMENT_REQUESTS(sender_id/receiver_id); WALLETS(wallet_id) -> TRANSACTIONS(sender_wallet_id/receiver_wallet_id); MERCHANTS(merchant_id) -> TRANSACTIONS(merchant_id).

Phase 2 - Database Creation and Data Preparation

1. **DDL:** Create WALLET_DB and all six tables using CREATE DATABASE and CREATE TABLE. PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints will be defined with the tables.
2. **DML:** Enter sample data using INSERT. UPDATE will be used when wallet balance or transaction status changes, while DELETE will be tested only where foreign-key relations are not affected.
3. **Data cleaning and constraints:** duplicate emails, phone numbers and account numbers will be prevented, required IDs will not be NULL, and invalid negative balance or transaction amount values will be restricted.
4. **NULL handling:** merchant_id will stay NULL for peer-to-peer transfers, and is_primary can stay at its default when not set. IS NULL / IS NOT NULL will be used to find such records.
5. **Basic queries:** SELECT, WHERE and ORDER BY will be used to display active wallets, successful transactions, payment requests by status and transactions by date.
6. **Aggregate functions:** COUNT() will calculate transactions per wallet, AVG() will calculate average transaction amount and SUM() will calculate total money spent per user.

Phase 3 - Data Querying and Analytical Operations

1. **Joins:** join USERS + WALLETS for balance details, WALLETS + TRANSACTIONS + MERCHANTS for spend reports, and USERS + PAYMENT_REQUESTS for request history.
2. **Nested queries and subqueries:** find users whose transaction count is above the average transaction count and users whose total spend is above the overall average.

3. **View:** create Wallet_Transaction_Summary to display user name, wallet balance, transaction count and total spend through one reusable query.
4. **Query execution and optimization:** use EXPLAIN on the wallet-transaction-merchant join. Indexes will be created on frequently searched columns such as TRANSACTIONS.sender_wallet_id, TRANSACTIONS.merchant_id and PAYMENT_REQUESTS.sender_id and the query plan will be checked again.
5. **Function:** create GetWalletTransactionCount(wallet_id). It will receive a wallet ID, count the matching records in TRANSACTIONS and return the total transactions of that wallet.
6. **Procedure:** create TransferFunds(sender_wallet_id, receiver_wallet_id, amount). It will accept the transfer details, debit the sender, credit the receiver and insert a new TRANSACTIONS record.

Functional Modules

Function / Task	Tables Used	How I will do it
Register user and wallet	USERS, WALLETS	I will add the user first and then create the wallet using user_id as the foreign key.
Link bank account	USERS, BANK_ACCOUNTS	I will store account number, IFSC code and bank name and link it to the user_id.
Transfer money between wallets	WALLETS, TRANSACTIONS	I will check sender balance, debit sender, credit receiver and log the transaction in one transaction block.
Pay a merchant	WALLETS, MERCHANTS, TRANSACTIONS	I will debit the user wallet, credit the merchant wallet and record the transaction with merchant_id.
Request money	USERS, PAYMENT_REQUESTS	I will store sender_id, receiver_id and amount, and update status when accepted or declined.
Flag suspicious activity	TRANSACTIONS	I will use triggers to flag transactions above a threshold and update the status field.
Generate summary reports	USERS, WALLETS, TRANSACTIONS, MERCHANTS	I will use joins, aggregates and the Wallet_Transaction_Summary view to prepare user and merchant summaries.

Phase 4 - Data Integrity and Transaction Handling

1. **Transaction handling:** when I transfer funds between wallets, I will start one transaction, debit the sender wallet and credit the receiver wallet. A SAVEPOINT will be used before the final update. COMMIT will be used if both operations succeed and ROLLBACK if an error occurs.
2. **Atomicity:** the debit and credit updates will be treated as one unit. If either operation fails, the complete transaction will be rolled back so the database does not remain partly updated.
3. **Concurrency case:** before confirming a transfer I will check that the sender wallet still has sufficient balance inside the transaction. This will help prevent the same balance from being spent in two simultaneous transactions.
4. **Integrity constraints:** foreign keys will ensure that TRANSACTIONS uses existing WALLETS and MERCHANTS, BANK_ACCOUNTS uses an existing USER, and PAYMENT_REQUESTS uses existing USERS.
5. **Final validation:** check for missing references, confirm that wallet balances match the sum of transactions, and run the basic queries, joins, subqueries, view, function, procedure and transaction cases to verify the database.

Outcome of the Project

1. A normalized relational database for digital wallet and payment records up to 3NF.
2. Practical implementation of DDL, DML, constraints, basic queries, aggregates, joins, subqueries, views, functions/procedures and transactions.
3. User, wallet, bank account, merchant, transaction and payment request information can be stored and retrieved through related tables.
4. The project will give hands-on understanding of data modelling, SQL querying, integrity and transaction handling using one complete database case study.